

Type-aware allocation and deallocation functions

Document #: P2719R2
Date: 2024-11-22
Project: Programming Language C++
Audience: Evolution
Reply-to: Louis Dionne
<ldionne@apple.com>
Oliver Hunt
<oliver@apple.com>

Contents

- 1 Introduction
- 2 Revision history
- 3 Motivation
 - 3.1 A concrete use case
- 4 Current behavior recap
- 5 Proposal
 - 5.1 Free function example
 - 5.2 In-class example
 - 5.3 Operator suppression example
 - 5.4 Template type allocation example
- 6 Proposed wording
 - 6.1 `basic.stc.dynamic.allocation`
 - 6.2 `basic.stc.dynamic.deallocation`
 - 6.3 `expr.new`
 - 6.4 `expr.delete`
- 7 Design choices and notes
 - 7.1 Design choice: `std::type_identity<T>` vs “raw” template argument
 - 7.2 Design choice: `std::type_identity<T>` vs `T*`
 - 7.3 Design choice: Location of `std::type_identity<T>`

- 7.4 Design choice: Templated type-aware operator delete is a *usual deallocation function*
- 7.5 Design choice: No support for type-aware destroying delete
- 7.6 Design choice: Dropped support for ADL
- 7.7 Interactions with `std::allocator<T>`
- 7.8 ODR implications and mismatched operator new / operator delete
- 7.9 Impact on the library

§ 1 Introduction

C++ currently provides two ways of customizing the creation of objects in new expressions. First, `operator new` can be provided as a static member function of a class, like `void* T::operator new`. If such a declaration is provided, an expression like `new T(...)` will use that allocation function. Otherwise, the global version of `operator new` can be replaced by users in a type-agnostic way, by implementing `void* operator new(size_t)` and its variants. A similar mechanism exists for *delete-expressions*.

This paper proposes an extension to *new-expressions* and *delete-expressions* to provide the concrete type being [de]allocated to the allocation functions. This is achieved via the use of an additional `std::type_identity<T>` tag argument that allows the provision of the concrete type to `operator new` and `operator delete`. In addition to providing valuable information to the allocator, this allows the creation of type-specific `operator new` and `operator delete` for types that cannot have intrusive class-scoped operators specified.

At a high level, this allows defining allocation and deallocation functions like:

```
void* operator new(std::type_identity<mylib::Foo>, std::size_t n) { ... }
void operator delete(std::type_identity<mylib::Foo>, void* ptr) { ... }
```

However, it also allows providing these functions for a family of types, which is where this feature becomes interesting:

```
template <class T>
    requires use_special_allocation_scheme<T>
void* operator new(std::type_identity<T>, std::size_t n) { ... }

template <class T>
    requires use_special_allocation_scheme<T>
void operator delete(std::type_identity<T>, void* ptr) { ... }
```

§ 2 Revision history

- R0: Initial version
- R1:

- Simplified the lookup mechanism by removing the need to perform ADL (based on implementation experience)
- Allowed `std::type_identity` parameter for in-class `T::operator new` for consistency
- Made `std::type_identity` as the first parameter
- Documented design choices and changes based on feedback and implementation experience
- R2:
 - Added the initial spec wording

§ 3 Motivation

Knowledge of the type being [de]allocated in a *new-expression* is necessary in order to achieve certain levels of flexibility when defining a custom allocation function. However, even when defining `T::operator new` in-class, the only information available to the implementation is the type declaring the operator, not the type being allocated. This results in developers various creative (often macro-based) mechanisms to define these allocation functions manually, or circumventing the language-provided allocation mechanisms entirely in order to track the allocated types.

However, in addition to these intrusive mechanisms being cumbersome and error-prone, they do not make it possible to customize how allocation is performed for types controlled by a third-party, or to customize allocation for an open set of types.

Beyond these issues, a common problem in we see in the wild is codebases overriding the global (and untyped) `operator new` via the usual link-time mechanism and running into problems because they really only intended for their custom `operator new` to be used within their own code, not by all the code in their process. For example, we've seen scenarios where multiple libraries attempt to replace the global `operator new` and end up with a complex ODR violation bug that depends on how the dynamic linker resolved weak definitions at load time – not very user friendly. By providing the concrete type information to allocators at compile time, it becomes possible for authors to override `operator new` for a family of types that they control without overriding it for the whole process, which is what they *actually* want.

§ 3.1 A concrete use case

A few years ago, Apple published [a blog post](#) explaining a technique used inside its kernel (XNU) to mitigate various exploits. At its core, the technique roughly consists in allocating objects of each type in a different bucket. By collocating all objects of the same type into the same region of memory, it becomes much harder for an attacker to exploit a type confusion vulnerability. Since its introduction in the kernel, this technique alone has been by far the most effective at mitigating type confusion vulnerabilities.

In a world where security is increasingly important, it may make sense for some code bases to adopt mitigation techniques such as this one. However, these techniques require a large-scale and *almost* system-wide customization of how allocation is performed while retaining type information, which is not supported by C++ today. While not sufficient in itself to make C++ safer, the change proposed in this paper is a necessary building block for technology such as the above which can greatly improve the security of C++ applications.

§ 4 Current behavior recap

Today, the compiler performs a `lookup` in the allocated type's class scope (for `T::operator new`), and then a lookup in the global scope (for `::operator new`) if the previous one failed. Once the name lookup has been done and the compiler has decided whether it was looking for `T::operator new` or `::operator new`, name lookup will not be done again even if the steps that follow were to fail. From here on, let's denote by `NEW` the set of candidates found by the name lookup process.

The compiler then performs `overload resolution` on that set of candidates using the language-specified optional implicit parameters, and if present any developer-provided placement arguments. It does so by assembling an argument list that depends on whether `T` has a new-extended alignment or not. For the sake of simplicity, assume that `T` does not have a new-extended alignment. The compiler starts by performing overload resolution as-if the following expression were used:

```
NEW(sizeof(T), args...)
```

If that succeeds, the compiler selects the overload that won. If it does not, the compiler performs overload resolution again as-if the following expression were used:

```
NEW(sizeof(T), std::align_val_t(alignof(T)), args...)
```

If that succeeds, the compiler selects the overload that won. If it does not, the program is ill-formed. For a type `T` that has new-extended alignment, the order of the two overload resolutions performed above is simply reversed.

Delete-expressions behave similarly, with lookup being performed in the context of the static type of the expression. The overload resolution process then works by preferring a destroying delete, followed by an aligned delete (if the type has new-extended alignment), followed by the usual `operator delete` (with or without a `size_t` parameter depending on whether the considered `operator delete` is a member function or not).

§ 5 Proposal

This proposal adds a new implicit tag argument of type `std::type_identity<T>` to `operator new` and `operator delete` that is incorporated into the existing overload resolution logic with a higher priority than existing implicit parameters. To avoid conflicts

with existing code, this parameter is placed as the first argument to the operator, preceding the size or subject pointer. To avoid the complexities of ADL, this proposal does not change any of the *name lookup* rules associated to *new* and *delete* expressions: it only changes the overload resolution that happens once a name has been found.

For the declaration of a type-aware [de]allocation operator to be valid, we explicitly require that the parameter be a (potentially dependent) specialization of `std::type_identity`, but not a fully dependent type. In other words, the compiler must be able to tell that the first parameter is of the form `std::type_identity<T>` at the time of parsing the declaration, but before the declaration has been instantiated in the case of a template. This is analogous to the current behavior where we require specific concrete types in the parameter list even in dependent contexts.

Once a set of candidate declarations has been found we perform the same prioritized overload resolution steps, only with the addition of `std::type_identity<T>`, with a higher priority than the existing size and alignment parameters. For illustration, here is how overload resolution changes (NEW is the set of candidates found by name lookup for `operator new`, and DELETE is the equivalent for `operator delete`).

If the user writes `new T(...)`, the compiler checks (in order):

Before	After
<pre>// T not overaligned NEW(sizeof(T)) NEW(sizeof(T), align_val_t{alignof(T)})</pre>	<pre>// T not overaligned NEW(type_identity<T>{}, sizeof(T)) NEW(type_identity<T>{}, sizeof(T), align_val_t{alignof(T)}) NEW(sizeof(T)) NEW(sizeof(T), align_val_t{alignof(T)})</pre>
<pre>// T overaligned NEW(sizeof(T), align_val_t{alignof(T)}) NEW(sizeof(T))</pre>	<pre>// T overaligned NEW(type_identity<T>{}, sizeof(T), align_val_t{alignof(T)}) NEW(type_identity<T>{}, sizeof(T)) NEW(sizeof(T), align_val_t{alignof(T)}) NEW(sizeof(T))</pre>

If the user writes `delete ptr`, the compiler checks (in order):

Before	After
<pre> // T not overaligned DELETE(T-or-Base*, destroying_delete_t{}, ...) DELETE((void*)ptr, sizeof(*ptr)) DELETE((void*)ptr) </pre>	<pre> // T not overaligned DELETE(T-or-Base*, destroying_delete_t{}, ...) DELETE(type_identity<T>{}, (void*)ptr, sizeof(T)) DELETE(type_identity<T>{}, (void*)ptr) DELETE((void*)ptr, sizeof(T)) DELETE((void*)ptr) </pre>
<pre> // T overaligned DELETE(T-or-Base*, destroying_delete_t{}, ...) DELETE((void*)ptr, sizeof(T), align_val_t{alignof(T)}) DELETE((void*)ptr, align_val_t{alignof(T)}) DELETE((void*)ptr) </pre>	<pre> // T overaligned DELETE(T-or-Base*, destroying_delete_t{}, ...) DELETE(type_identity<T>{}, (void*)ptr, sizeof(T), align_val_t{alignof(T)}) DELETE(type_identity<T>{}, (void*)ptr, align_val_t{alignof(T)}) DELETE(type_identity<T>{}, (void*)ptr) DELETE((void*)ptr, sizeof(T), align_val_t{alignof(T)}) DELETE((void*)ptr, align_val_t{alignof(T)}) DELETE((void*)ptr) </pre>

If multiple candidates match a given set of parameters, candidate prioritisation and selection is performed according to usual rules for overload resolution.

When a constructor throws an exception, a call to `operator delete` is made to clean up. Overload resolution for this call remains essentially the same, the only difference being that the selected `operator delete` must have the same type-awareness as the preceding `operator new` or the program is considered ill-formed.

For clarity, in types with virtual destructors, `operator delete` is resolved using the destructor's class as the type being deallocated (this matches the existing semantics of being equivalent to performing `delete this` in the context of the class's non virtual destructor).

§ 5.1 Free function example

```

struct SingleClass { };
struct UnrelatedClass { };
struct BaseClass { };
struct SubClass1 : BaseClass { };
struct SubClass2 : BaseClass { };
struct SubClass3 : BaseClass { };

```

```

void* operator new(std::type_identity<SingleClass>, std::size_t); // (1)
template <typename T> void* operator new(std::type_identity<T>,
std::size_t); // (2)

template <std::derived_from<BaseClass> T>
void* operator new(std::type_identity<T>, std::size_t); // (3)
void* operator new(std::type_identity<SubClass2>, std::size_t); // (4)
void* operator new(std::type_identity<SubClass3>, std::size_t) = delete;
// (5)

struct SubClass4 : BaseClass {
    void *operator new(size_t); // (6)
};

void f() {
    new SingleClass(); // calls (1)
    new UnrelatedClass(); // calls (2)
    new BaseClass(); // calls (3) with T=BaseClass
    new SubClass1(); // calls (3) with T=SubClass1
    new SubClass2(); // calls (4)
    new SubClass3(); // resolves (5) reports error due to deleted
operator
    new SubClass4(); // calls (6) as the class scoped operator wins
    new int(); // calls (2) with T=int
}

```

Note: The above is for illustrative purposes only: it is a bad idea to provide a fully unconstrained type-aware `operator new`.

§ 5.2 In-class example

```

// In-class operator
class SubClass1;
struct BaseClass {
    template <typename T>
    void* operator new(std::type_identity<T>, std::size_t); // (1)
    void* operator new(std::type_identity<SubClass1>, std::size_t); // (2)
};

struct SubClass1 : BaseClass { };
struct SubClass2 : BaseClass { };
struct SubClass3 : BaseClass {
    void *operator new(std::size_t); // (3)
};
struct SubClass4 : BaseClass {
    template <typename T>
    void *operator new(std::type_identity<T>, std::size_t); // (4)
};

void f() {
    new BaseClass; // calls (1) with T=BaseClass
    new SubClass1(); // calls (2)
    new SubClass2(); // calls (1) with T=SubClass2
    new SubClass3(); // calls (3)
    new SubClass4(); // calls (4) with T=SubClass4
}

```

```

::new BaseClass();           // ignores in-class operators and uses
                             appropriate global operator
}

```

§ 5.3 Operator suppression example

There are many cases where projects may not want types to be allocated and deallocated via `new` and `delete` operators. Doing so today requires injecting operators into the relevant types, which often results in extensive use of macros. This proposal allows constraint based selection of target types, and as such can be leveraged to specify deleted operators, and so automatically prevent their use e.g.

```

template <typename T> concept SelectionConstraint = ...;
template <SelectionConstraint T> void *operator new(std::type_identity<T>,
std::size_t) = delete;

...
template <SelectionConstraint T> void operator
delete(std::type_identity<T>, void *) = delete;
...

```

§ 5.4 Template type allocation example

The template arguments to a type aware operator `new` or `delete` are not required to be directly applied to `std::type_identity`, but are simply available for usual template deduction, so a type aware allocation function can be defined to operate over a template type, e.g.

```

template <typename T, int N>
struct MyArrayType {
    ...
};
template <typename T, int N>
void *operator new(std::type_identity<MyArrayType<T, N>>, size_t, ...) {
    ...
}
...
//      calls      the      above      operator      new<int,      5>
//      (std::type_identity<MyArrayType<int, 5>>, ...)
auto A = new MyArrayType<int, 5>;

```

§ 6 Proposed wording

§ 6.1 basic.stc.dynamic.allocation

Updates for 6.7.5.5.2 [\[basic.stc.dynamic.allocation\]](#)

- 1 An allocation function that is not a class member function shall belong to the global scope and not have a name with internal linkage. The return type shall be `void*`. An allocation function shall have at least one parameter. If the first parameter is of type `std::type_identity<T>` for some type `T` there shall be

at least two parameters; the first parameter is called the type-identity parameter, the second parameter is called the size parameter. Otherwise, the first parameter is called the size parameter. The ~~first~~size parameter shall have type `std::size_t` (17.2 [support.types]). ~~The first~~Both the type-identity, if present, and the size parameters shall not have an associated default argument (9.3.4.7 [dcl.fct.default]). The value of the ~~first~~size parameter is interpreted as the requested size of the allocation. An allocation function can be a function template. Such a template shall declare its return type and ~~first~~size parameter as specified above, and the type-identity parameter shall be a specialization of `std::type_identity` (that is, template argument types shall not be used in the return type and ~~first~~size parameter type, and if present in the declaration of the type-identity parameter they only be used in the template argument to `std::type_identity`). Allocation function templates shall have two or more parameters.

§ 6.2 basic.stc.dynamic.deallocation

Updates for 6.7.5.5.3 [basic.stc.dynamic.deallocation]

- 2 A deallocation function is a destroying operator delete if it has at least two parameters and its second parameter is of type `std::destroying_delete_t`. A destroying operator delete shall be a class member function named operator delete.
- 3 Each deallocation function shall return void. A deallocation function shall have at least one parameter. If the first parameter is a of type `std::type_identity<T>` for some type T, there shall be at least two parameters; the first parameter is called the type-identity parameter, the second parameter is called the object parameter. Otherwise, the first parameter is called the object parameter. If the function is a destroying operator delete declared in class type C, the type of its ~~first~~object parameter shall be `C*`; otherwise, the type of its ~~first~~object parameter shall be `void*`. A deallocation function may have more than one parameter. A usual deallocation function is a deallocation function whose parameters after the ~~first~~object parameter are
 - (3.1) — optionally, a parameter of type `std::destroying_delete_t`, then
 - (3.2) — optionally, a parameter of type `std::size_t`, then
 - (3.3) — optionally, a parameter of type `std::align_val_t`.

A destroying operator delete shall be a usual deallocation function. A deallocation function may be an instance of a function template. Neither the ~~first~~object parameter nor the return type shall depend on a template parameter. A deallocation function template shall have two or more function parameters. A template instance ~~is never a usual deallocation function, regardless of its signature.~~is only a usual deallocation function if it has at least two parameters,

the first parameter is a type-identity, and the type-identity is the only dependent parameter (i.e. the type-identity parameter may be of type `std::type_identity<T>` where T is a dependent type).

§ 6.3 `expr.new`

Updates to 7.6.2.8 [`expr.new`]

20 Overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. If the type of the allocated object has new-extended alignment, the next argument is the type's alignment, and has type `std::align_val_t`. If the new-placement syntax is used, the initializer-clauses in its expression-list are the succeeding arguments. If no matching function is found then

- (20.1) — if the allocated object type has new-extended alignment, the alignment argument is removed from the argument list;
- (20.2) — otherwise, an argument that is the type's alignment and has type `std::align_val_t` is added into the argument list immediately after the first argument;

and then overload resolution is performed again.

[*Note:* For Reviewers: in the existing text the type of the allocated object for both `new T` and `new T[expr]` is T based on how class scope is determined, this does not seem very clear, and has significant semantic difference: `type_identity<T>` vs `type_identity<T[]>` — *end note*]

20 The type-identity is a default-initialized object of type `std::type_identity<T>`, where T is the type of the allocated object.

[*Note:* For Reviewers: The following changes attempt to restructure the existing wording to reuse the alignment “correction” performed. We are not sure what the correcting notation for such changes is. — *end note*]

20->21 Overload resolution is performed on a function call by assembling an argument list.

[*Note:* For Reviewers: these points exist, but are being inset and numbered/put into a sub-list — *end note*]

- (+ 21.1) — The first argument is the type-identity, the second~~The first~~ argument is the amount of space requested, and has type `std::size_t`.
- (+ 21.2) — If the type of the allocated object has new-extended alignment, the next argument is the type's alignment, and has type `std::align_val_t`. If the

new-placement syntax is used, the initializer-clauses in its expression-list are the succeeding arguments. If no matching function is found then

- (20.1 -> 21.2.1) — if the allocated object type has new-extended alignment, the alignment argument is removed from the argument list;
- (20.2 -> 21.2.2) — otherwise, an argument that is the type's alignment and has type `std::align_val_t` is added into the argument list immediately after the first argument;
- (+ 21.3) — and then overload resolution is performed again.

[For Reviewers: end the added inset + new sublist]

- (21.4) — If no matching function has been found then paragraphs [21.3]-[21.4] are repeated starting from an argument list consisting of a single parameter that is the amount of space requested, and has type `std::size_t`.

29 A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations (9.3.4.6 [dcl.fct]), all parameter types except ~~the first~~the size and object are identical.

30 If a new-expression calls a deallocation function it passes the type-identity as the first argument if it was passed to the allocation function, and the value returned from the allocation function call as the ~~first argument~~argument for the object parameter of type `void *`.

§ 6.4 **expr.delete**

9 The type-identity is an object with an unspecified value of type `std::type_identity` where T is the static type of the operand

[*Note:* For Review: is this sufficient to say that given `X *Obj` for `delete Obj` and `delete [] Obj` will have a type-identity of type `std::type_identity<X>?` — *end note*]

10 Construct a set of candidate deallocation functions

- (10.1) — All found declarations for which the first parameter is of a pointer type [*Note:* either `void *`, or a `C*` if a destroying delete for some type C — *end note*] or the same type as the type-identity are added to the set of candidate functions.

- (10.2) — For each function template found, function template argument deduction (13.10.3 [temp.deduct]) is performed using the type of the type-identity as the first parameter, and the type of each subsequent parameter in the candidate function template. If argument deduction and checking succeeds, the deduced template-arguments are used to synthesize the

declaration of a single function template specialization. If the first parameter of the synthesized function is of the same type as the type-identity, it is added to the set of candidates.

[*Note:* For Review: It is unclear if this last constraint needs to be stated explicitly as it would require a non-standard declaration of `std::type_identity` which I think would be UB? — *end note*]

(10.2) — If multiple candidate functions have the same type, all candidates other than the candidate that is the best viable function as described in 12.2.4 [over.match.best] are removed from the set of candidates.

11 The deallocation function to be called is selected from the candidate set as follows:

(9.1->11.1) — If any of the deallocation functions is a destroying operator delete, all deallocation functions that are not destroying operator deletes are eliminated from further consideration.

(11.2) — If any of the deallocation candidates have a first parameter of the same type as the type-identity, all deallocation functions that do not have a first parameter of that type are eliminated from further consideration.

(9.2->11.3) — If the type has new-extended alignment, a function with a parameter of type `std::align_val_t` is preferred; otherwise a function without such a parameter is preferred. If any preferred functions are found, all non-preferred functions are eliminated from further consideration.

...

10->12 If the first parameter of the selected deallocation function is the same type as the type-identity, the type-identity is passed as the first argument to the deallocation call. For a single-object delete expression, the deleted object is the object *A* pointed to by the operand if the static type of *A* does not have a virtual destructor, and the most-derived object of *A* otherwise. ...

§ 7 Design choices and notes

§ 7.1 Design choice: `std::type_identity<T>` vs “raw” template argument

In an earlier draft, this paper was proposing the following (seemingly simpler) mechanism. Instead of using `std::type_identity<T>` as a tag, the compiler would search as per the following expression:

```
operator new<T>(sizeof(T), args...)
```

The only difference here is that we're passing the type being allocated directly as a template argument instead of using a `std::type_identity<T>` tag parameter. Unfortunately, this has a number of problems, the most significant being that it's not possible to distinguish the newly-introduced type-aware operator from existing template `operator new` and `operator delete` declarations.

For example, a valid `operator new` declaration today would be:

```
template <class ...Args>
void* operator new(std::size_t, Args...);
```

Hence, an expression like `new (42) int(3)` which would result in a call like `operator new<int>(sizeof(int), 42)` could result in this operator being called with a meaning that isn't clear – is it a type-aware (placement) operator or a type-unaware placement operator? This also means that existing and legal operators could start being called in code bases that don't expect it, which is problematic.

Beyond that being confusing for users, this also creates a legitimate problem for the compiler since the resolution of `new` and `delete` expressions is based on checking various forms of the operators using different priorities. In order for this to make sense, the compiler has to be able to know exactly what “category” of operator a declaration falls in, so it can perform overload resolution at each priority on the right candidates.

Finally, when a constructor in a *new-expression* throws an exception, an `operator delete` that must be a *usual deallocation function* gets called to clean up. If there is no matching usual deallocation function, no cleanup is performed. Using a template parameter instead of a tag argument could lead to code where no cleanup happened to now find a valid usual deallocation function and perform a cleanup.

Taken together we believe these issues warrant the use of an explicit tag parameter.

§ 7.2 Design choice: `std::type_identity<T>` vs `T*`

This proposal uses `std::type_identity<T>` as a tag argument rather than passing a first argument of type `T*`. At first sight, passing `T*` as a first tag argument seems to simplify the proposal and decouple the compiler from the standard library.

However, this approach hides an array of subtle problems that are avoided through the use of `std::type_identity`.

§ 7.2.1 Problems with the value being passed

The first problem is the value being passed as the tag parameter. Given operator signatures of the form

```
template <class T> void *operator new(T*, size_t);
template <class T> void operator delete(T*, void*);
```

Under the hood, the compiler could perform calls like

```
// T* ptr = new T(...)
operator new<T>((T*)nullptr, sizeof(T));

// delete ptr
operator delete<T>((T*)nullptr, ptr);
```

A developer could reasonably be assumed to know that the tag parameter to `operator new` can't be anything but a null pointer. However, for `operator delete` we can be assured that people will be confused about receiving two pointer parameters, where the explicitly typed parameter is `nullptr`. Also note that we cannot pass the object pointer through that parameter as `operator delete` is called after the object has been destroyed. Passing the memory to be deallocated through a typed pointer is an incitation to use that memory as a T object, which would be undefined behavior.

§ 7.2.2 Silent conversions to base types

A scenario we have discussed is developers wishing to provide custom [de]allocation operators for a whole class hierarchy. When using a typed pointer as the tag, this would be written as:

```
struct Base { };
void* operator new(Base*, std::size_t);
void operator delete(Base*, void*);
```

This operator would then also match any derived types of Base, which may or may not be intended. If not intended, the conversion from `Derived*` to `Base*` would be entirely silent and may not be noticed. Furthermore, this would basically defeat the purpose of providing type knowledge to the allocator, since only the type of the base class would be known. We believe that the correct way of implementing an operator for a hierarchy is this:

```
struct Base {
    template <class T>
        void* operator new(std::type_identity<T>, std::size_t); // T is the
                                actual type being allocated

    template <class T>
        void operator delete(std::type_identity<T>, void*);
};
```

Or alternatively, in the global namespace:

```
template <std::derived_from<Base> T>
void* operator new(std::type_identity<T>, std::size_t);

template <std::derived_from<Base> T>
void operator delete(std::type_identity<T>, void*);
```

§ 7.2.3 Performance considerations

There is a fundamental difference between `T*` and `std::type_identity<T>` in that `T*` is a type that has an actual value and size, whereas `std::type_identity` is a zero sized

record. This difference means that the T^* model results in an additional parameter being required in the generated code, whereas the zero sized `type_identity` parameter does not exist in the majority of calling conventions. In principle this difference should be minor for templated operators as they are typically inlined and so the calling convention is not relevant, but for non-template definitions the implementation can be out of line, and so the difference may matter.

For all of these reasons, we believe that a tag type like `std::type_identity` is the right design choice.

§ 7.3 Design choice: Location of `std::type_identity<T>`

When writing this paper, we went back and forth of the order of arguments. This version of the paper proposes:

```
operator new(std::type_identity<T>, std::size_t, placement-args...)
operator new(std::type_identity<T>, std::size_t, std::align_val_t,
            placement-args...)

operator delete(std::type_identity<T>, void*)
operator delete(std::type_identity<T>, void*, std::size_t)
operator delete(std::type_identity<T>, void*, std::size_t,
               std::align_val_t)
```

Another approach would be:

```
operator new(std::size_t, std::type_identity<T>, placement-args...)
operator new(std::size_t, std::align_val_t, std::type_identity<T>,
            placement-args...)

operator delete(void*, std::type_identity<T>)
operator delete(void*, std::size_t, std::type_identity<T>)
operator delete(void*, std::size_t, std::align_val_t,
               std::type_identity<T>)
```

The existing specification allows for the existence of template (including variadic template) declarations of `operator new` and `delete`, and this functionality is used in existing code bases. This leads to problems compiling real world code where overload resolution will allow selection of a non-SFINAE-safe declaration and subsequently break during compilation.

Placing the tag argument first ensures that no existing operator definition can match, and so we are guaranteed to be free from conflicts.

§ 7.4 Design choice: Templated `type-aware operator delete` is a *usual deallocation function*

Allowing `type-aware operator delete` does require changes to the definition of usual deallocation functions, but the changes are conceptually simple and the cost of not supporting this case is extremely high.

In the current specification, we place very tight requirements on what an `operator delete` declaration can look like in order to be considered a *usual deallocation function*. The reason this definition previously disallowed function templates is that all of the implicit parameters are monomorphic types. That restriction made sense previously.

However, this proposal introduces a new form of the operators for which it is correct (even expected) to be a function template. To that end, we allow a templated `operator delete` to be considered a usual deallocation function, as long as the only dependently-typed parameter is the first `std::type_identity<T>` parameter. To our minds, these semantics match the “intent” of the restrictions already in place for the other implicit parameters like `std::align_val_t`.

The cost of not allowing a templated type-aware `operator delete` as a usual deallocation function is very high, as it functionally prohibits the use of type-aware allocation operators in any environment that requires the ability to clean up after a constructor has thrown an exception.

§ 7.5 Design choice: No support for type-aware destroying delete

We have decided not to support type-aware destroying delete as we believe it creates a user hazard. At a technical level there is no additional complexity in supporting type-aware destroying delete, but the resulting semantics seem likely to cause a lot of confusion. For example, given this hypothetical declaration:

```
struct Foo {
    ...
    template <class T>
        void operator delete(std::type_identity<T>,      Foo*,
                             std::destroying_delete_t);
};

struct Bar : Foo { };

void f(Foo* foo) {
    delete foo; // calls Foo::operator delete<Foo>
}

void g(Bar *bar) {
    delete bar; // calls Foo::operator delete<Bar>
}
```

To a user this appears to be doing what they expect. However, consider the following:

```
struct Oops : Bar { };

void h(Oops *oops) {
    g(oops); // calls Foo::operator delete<Bar> from within g()
}
```

By design, destroying delete does not perform any polymorphic dispatch, and as a result the type being passed to the operator is not be the dynamic type of the object being destroyed, but rather its static type. As a result, basic functionality will appear to work correctly from the user’s point of view when in reality the rules are much subtler than they seem.

Given that the design intent of destroying delete is for users to manage destruction and dispatching manually, we believe that adding type-awareness to destroying delete will add little value while creating the potential for confusion, so we decided not to do it.

§ 7.6 Design choice: Dropped support for ADL

The initial proposal allowed the specification of type-aware operators in namespaces that would then be resolved via ADL. Upon further consideration, this introduces a number of challenges that are difficult to resolve robustly. As a result, we have dropped support for namespace-scope operator declarations and removed the use of ADL from the proposal.

The first problem is that ADL would be based on the type of all arguments passed to `operator new`, including placement arguments. While this is not a problem for `operator new` itself, `operator delete` does not get the same placement arguments, which would potentially change the set of associated namespaces used to resolve `new` and `delete`.

One of our original motivations for allowing namespace-scoped operators was to simplify the task of providing operators for a whole library. However, since ADL is so viral, the set of associated namespaces can easily grow unintentionally (e.g. `new lib1::Foo<lib2::Bar>(...)`), which means that developers would have to appropriately constrain their type-aware operators anyway. In other words, we believe that a declaration like this would never have been a good idea in the first place:

```
namespace lib {  
    // intent: override for all types in this namespace  
    template <class T>  
    void* operator new(std::type_identity<T>, std::size_t);  
}
```

There are too many ways in which an unconstrained declaration like this can break, including an unexpected set of associated namespaces or even a mere `using namespace lib;`.

Given the need to constrain a type-aware operator anyway, we believe that allowing namespace-scoped operators is merely a nice-to-have but not something that we need fundamentally. Furthermore, adding this capability to the language could always be pursued as a separate proposal since that concern can be tackled orthogonally. For example, a special ADL lookup could be done based solely on the dynamic type being [de]allocated.

Since this adds complexity to the proposal and implementation and doesn’t provide great value, we are not pursuing it as part of this proposal.

§ 7.7 Interactions with `std::allocator<T>`

Today, `std::allocator<T>::allocate` is specified to call `::operator new(std::size_t)` explicitly. Even if `T::operator new` exists, `std::allocator<T>` will not attempt to call it. We view this as a defect in the current Standard since `std::allocator<T>` could instead select the same operator that would be called in an expression like `new T(...)` (without the constructor call, obviously).

This doesn't have an interaction with our proposal, except for making `std::allocator<T>`'s behavior a bit more unfortunate than it already is today. Indeed, users may rightly expect that `std::allocator<T>` will call their type-aware `operator new` when in reality that won't be the case.

Since this deception already exists for `T::operator new`, we do not attempt to change `std::allocator<T>`'s behavior in this proposal. However, the authors are willing to investigate fixing this issue as a separate proposal, which will certainly present its own set of challenges (e.g. constant evaluation).

§ 7.8 ODR implications and mismatched `operator new / operator delete`

A concern that was raised in St-Louis was that this proposal would increase the likelihood of ODR violation caused by different declarations of `operator new/operator delete` being used in different TUs. For example, one TU would get `lib1::operator new` and another TU would use `lib2::operator delete` due to e.g. a different set of headers being included. Note that the exact same issue also applies to every other operator that is commonly used via ADL (like `operator+`), except that many such ODR violations may end up being more benign than a mismatched `new/delete`.

First, we believe that the only way to avoid this issue (in general) is to properly constrain templated declarations, and nothing can prevent users from doing that incorrectly. However, since this proposal has dropped the ADL lookup, declarations of type-aware operators must now be in-class or global. This greatly simplifies the selection of an operator, which should make it harder for users to unexpectedly define an insufficiently constrained operator without immediately getting a compilation error.

Furthermore, without ADL lookup, the ODR implications of this proposal are exactly the same as the existing ODR implications of user-defined placement new operators, which can be templates.

§ 7.9 Impact on the library

This proposal does not have any impact on the library, since this only tweaks the search process performed by the compiler when it evaluates a new-expression and a delete-expression. In particular, we do not propose adding new type-aware free function

`operator new` variants in the standard library at this time, although this could be investigated in the future.